THE RELAY — ENGINEERING REPORT

Research-paper assistant | 11 November 2026

1. What I built and why

I built a Python research-paper assistant over exactly three versioned arXiv PDFs: Attention Is All You Need (1706.03762v7), Moshi (2410.00037v1), and Dynamic Chunking (2507.07955v2). The 120-page corpus is extracted, token-chunked with overlap, and indexed with BM25. The pipeline retrieves evidence, decides whether context is relevant, generates a cited answer through Groq, validates its structure, and retries or refuses when appropriate. This makes retrieval, generation, latency, and failure behavior observable.

2. The real service and my stand-in

Real service: Groq model *openai/gpt-oss-20b*; 14 real requests (five pipeline/baseline calls and nine answer-quality calls). Captured headers confirmed RPD=1,000 and TPM=8,000. Two real 429 events were TPM-related; observed Retry-After values were about 1.13 s and 16.23 s. **Stand-in:** local streaming OpenAI-compatible service simulated slow responses, 503 failures, 429 quota refusals, flaky recovery, timeouts, and concurrency. Tested settings included max in-flight=5, RPM=10 (halved to 5), and daily request budget=5.

Table 1. Rate limits and evidence

Limit	Groq free tier / evidence	Response when hit	Confidence
RPM	30 published for model	429 on RPM exhaustion	Published; not observed
RPD	1,000 published; header observed	429; quota reset	Header + published
TPM	8,000 published; header observed	429; Retry-After/reset	Header + published
TPD	200,000 published for model	429 on daily token quota	Published; not observed
Requests in flight	Not published / not confirmed	Unknown for Groq	Unknown
Stand-in concurrency	5 configured locally	503 at capacity	Directly tested

Table 2a. Latency by traffic scenario (successful requests only)

Traffic	N	Duration	Offered rate	TTFT p50 / p95	Total p50 / p95	Source
Steady	10	9.435 s	2 req/s	1,955 / 2,348 ms	4,948 / 5,939 ms	Stand-in
Burst	12	5.975 s	12 in 32 ms	1,580 / 2,284 ms	3,978 / 5,749 ms	Stand-in
Overload	20	6.150 s	20 req/s cfg.	1,608 / 2,275 ms	4,006 / 5,739 ms	Stand-in

Table 2b. Outcomes and capacity

Scenario	N	Success	429	5xx/error	Timeout	Retries/req	Peak in-flight
Steady	10	70%	0	3	0	0	6
Burst	12	41.7%	0	7	0	0	12
Overload	20	25%	0	15	0	0	6
Halved RPM	8	62.5%	3	0	0	0	8 launched
Daily budget	8	62.5%	3	0	0	0	8 launched

Load profile: steady 10 requests over 9.435 s (2 req/s); burst 12 launched within 32 ms (peak 12 in-flight); overload 20 requests over 6.150 s (about 20 req/s configured; peak 6 in-flight). The stand-in rejects excess concurrency rather than maintaining a queue. Quota tests launched eight requests together. Percentiles use small success samples and are low-confidence.

THE RELAY — RESULTS, LIMITATIONS & REFLECTION

3. Retrieval quality and answer quality

Table 3. Three manually graded questions per setting. Latencies are ms; cost is estimated per 1,000 answers.

Chunk / overlap	Q	Correct/cited	Wrong	Correct refusal	Tokens/answer	TTFT p50/p95	Total p50/p95	Cost/1k
300 / 30	3	33.3%	66.7%	0%	2,395	1,095 / 1,102	1,380 / 1,475	$0.3182
500 / 50	3	66.7%	33.3%	0%	3,081	1,085 / 1,328	1,418 / 1,746	$0.3749
700 / 70	3	100%	0%	0%	3,931	944 / 1,219	1,302 / 1,650	$0.4239

Retrieval: quality versus cost. All three chunk settings retrieved a supporting page in the top five for all nine audit questions. Rank-1 supporting-page rate rose from 55.6% (300/30) to 66.7% (500/50) to 77.8% (700/70). In the smaller answer evaluation (three questions per setting), 700/70 scored 3/3 but used about 64% more tokens per answer than 300/30. These tiny samples are directional, not conclusive. Cost estimates use published list prices ($0.075/M input and $0.30/M output tokens), not billed spend.

How this library can mislead a naive retriever. BM25 can rank lexical matches that are not evidence: Moshi Q04 ranked an unrelated synthetic-conversation appendix page first while the relevant method page ranked second; Dynamic Chunking Q09 ranked a taxonomy page first instead of the more useful discussion. Multi-head-attention evidence appeared below rank one for one Transformer query. OCR-like or appendix content can look relevant by term overlap.

4. Where the time goes

BM25 retrieval took roughly 0.64–3.12 ms in the nine-question retrieval audit; negligible beside generation. Five real pipeline calls had TTFT 691–1,292 ms and total time 935–1,538 ms. In stand-in load tests, successful responses typically took about 4.0–4.9 s total, with TTFT about 1.6–2.0 s. Generation and streaming dominate end-to-end latency.

5. When a limit is hit

In the stand-in, a 12-request burst reached 12 in-flight against a cap of five; seven failed with 503. At RPM=5 and daily budget=5, eight simultaneous attempts yielded five successes and three 429s. Rejected attempts also counted toward the stand-in daily-use counter. Real Groq 429s were TPM-related, not demonstrated RPM/RPD exhaustion.

6. Retries

Bounded retries (two maximum) use exponential backoff (0.5 s, then 1 s) unless Retry-After is supplied. Flaky mode recovered on attempt 3 (2 retries; 3.64 s). Repeated 503 ended after 3 attempts (1.64 s); 429 respected two 5 s waits (10.19 s); a stream timeout retried once and ended after 4.68 s. The timeout test ended as timeout despite HTTP 200 headers. Retrying increases latency and can duplicate work if generation completed but the response was lost.

7. What the person sees

The assistant streams generated text and supplies paper/page citations. A deterministic validator checks answer structure and whether cited sources belong to retrieved context; it is not a semantic fact-checker. Invalid or unsupported answers can be retried or refused. In nine manually graded answers, six were correct and three wrong.

8. What I am unsure about, left out, and learned

The answer-quality comparison uses only Q02, Q04, and Q08 (three per chunk setting); grades were manual. Small samples make p95/p99 unstable. Six older baseline rows have missing timestamps and remain blank. Groq concurrency and exact recovery behavior were not established. The stand-in is not a production replica, and no real high-load test was attempted. I learned to separate HTTP status from stream completion, verify rate-limit header semantics, and retain raw measurements rather than infer missing values.

Your feedback on Vaani. Vaani was helpful for breaking a complex assignment into manageable steps, debugging code, and structuring latency, quota, and retry experiments. The workflow also showed that AI suggestions need independent checking against code, documentation, and measured outputs. Clearer warnings about assumptions and stronger support for validating experimental evidence would make it more useful.

How I worked with the assistant. I used the assistant for stepwise implementation guidance, debugging, experiment design, and report structure, then ran commands locally and supplied outputs. I checked and corrected a mistaken interpretation of the Groq request-limit header (RPD, not RPM), and we caught the HTTP-200/stream-timeout case. I retained measured results, manually graded answers, and left six unknown timestamps blank rather than inventing them. The assistant proposed code and analysis; local runs and final verification supplied the evidence.

Sources: Groq rate limits and model documentation: https://console.groq.com/docs/rate-limits and https://console.groq.com/docs/models. Project figures are from local CSV outputs and experiment logs. Missing timestamps and unknown Groq concurrency were not inferred.